华清远见 HQYJ.COM

贝叶斯

回顾

在本节中，我们将回顾贝叶斯方法的基本概念和应用。贝叶斯方法是一种统计推断的方法，它利用贝叶斯定理来更新我们对事件概率的信念。这种方法在机器学习和数据分析中得到广泛应用，特别是在分类和概率估计问题上。

数据集介绍

我们使用的数据集是鸢尾花数据集UCI Machine Learning Repository，包含150个样本和4个特征共三种类别。

- 花萼长度（厘米）
- 花萼宽度（厘米）
- 花瓣长度（厘米）
- 花瓣宽度（厘米）
- 类别：
  -- 山鸢尾
  -- 变色鸢尾
  -- 弗吉尼亚鸢尾
  这个数据集用于分类任务，我们将利用贝叶斯方法来解决其中的问题。

代码分析

读取数据集

首先，我们需要读取数据集，为了方便还可以添加列名，通过`pandas`库读取存储在'../dataset/iris.data'文件中的数据集。

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

```python
# 假设数据保存在iris.data文件中
dataset = pd.read_csv('../dataset/iris.data')

# 添加列名
column_names = ['sepal length', 'sepal width', 'petal length', 'petal width', 'class']
dataset.columns = column_names

# 检查数据的前几行，确保数据加载正确
print(dataset.head())
```

数据处理

构建数据集，选取特征x和目标y

```python
# 假设特征列为X，目标变量列为y
X = dataset.drop('class', axis=1)
y = dataset['class']
```

划分数据集

```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

进行标准化处理

```python
# 使用标准化进行特征缩放(本例中的特征都是离散型实际上可以不进行归一化)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

模型训练

首先初始化分类器然后进行训练

```python
# 初始化朴素贝叶斯分类器
nb_classifier = GaussianNB()

# 在训练集上训练模型
nb_classifier.fit(X_train, y_train)
```

华清远见元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

模型评估

模型训练完成后，我们对模型的性能进行评估。我们使用测试集进行预测，并计算准确率、混淆矩阵和分类报告等指标来了解模型的表现。

```
# 在测试集上进行预测
y_pred = nb_classifier.predict(X_test)

# 评估模型性能
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print(f'Confusion Matrix:\n{conf_matrix}')
print(f'Classification Report:\n{class_report}')
```

```
Accuracy: 0.9066666666666666
Confusion Matrix:
[[29  0  0]
 [ 0 19  5]
 [ 0  2 20]]
Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        29
Iris-versicolor       0.90      0.79      0.84        24
 Iris-virginica       0.80      0.91      0.85        22

       accuracy                           0.91        75
      macro avg       0.90      0.90      0.90        75
   weighted avg       0.91      0.91      0.91        75
```

我们可以分析这个评估结果：

准确度 (Accuracy):

华清远见|元宇宙实验中心 yyzlab.com.cn

华清远见 HQYJ.COM

- Accuracy: 0.91
- 准确度是正确预测的样本数与总样本数之比。在这个情况下，模型的准确度为91%，表示91%的样本被正确分类。

混淆矩阵（Classification Report）：

混淆矩阵显示了模型的分类情况。对角线上的元素表示正确分类的样本数，而非对角线上的元素表示错误分类的样本数。例如，29个类别一被正确分类，而5个类别二被错误分类成类别三。

分类报告 (Classification Report):
分类报告提供了更详细的性能指标，包括精度(precision)、召回率(recall)、F1分数(f1-score)等。

- 对于“Iris-setosa”类别，精度为100%，召回率为100%，F1分数为100%。
- 对于“Iris-versicolor”类别，精度为90%，召回率为79%，F1分数为84%。
- 宏平均 (macro avg) 和加权平均 (weighted avg) 提供了整体性能的总结。在这个情况下，宏平均的精确度分数为90%，加权平均的精确度分数为91%；宏平均的召回率分数为90%，加权平均的召回率分数为91%；宏平均的F1分数为90%，加权平均的F1分数为91%。这些指标是所有类别性能的平均值。

可视化

最后，我们使用t-SNE对SVM模型的决策函数结果进行降维，创建散点图进行可视化，以便更直观地理解模型在二维空间中的预测结果。通过这个可视化图，你可以观察贝叶斯分类器在二维空间中的预测分布情况。不同颜色的点表示不同的类别，你可以看到它们在t-SNE空间中的相对位置关系。这有助于判断模型的分类效果和数据在降维空间中的分布情况

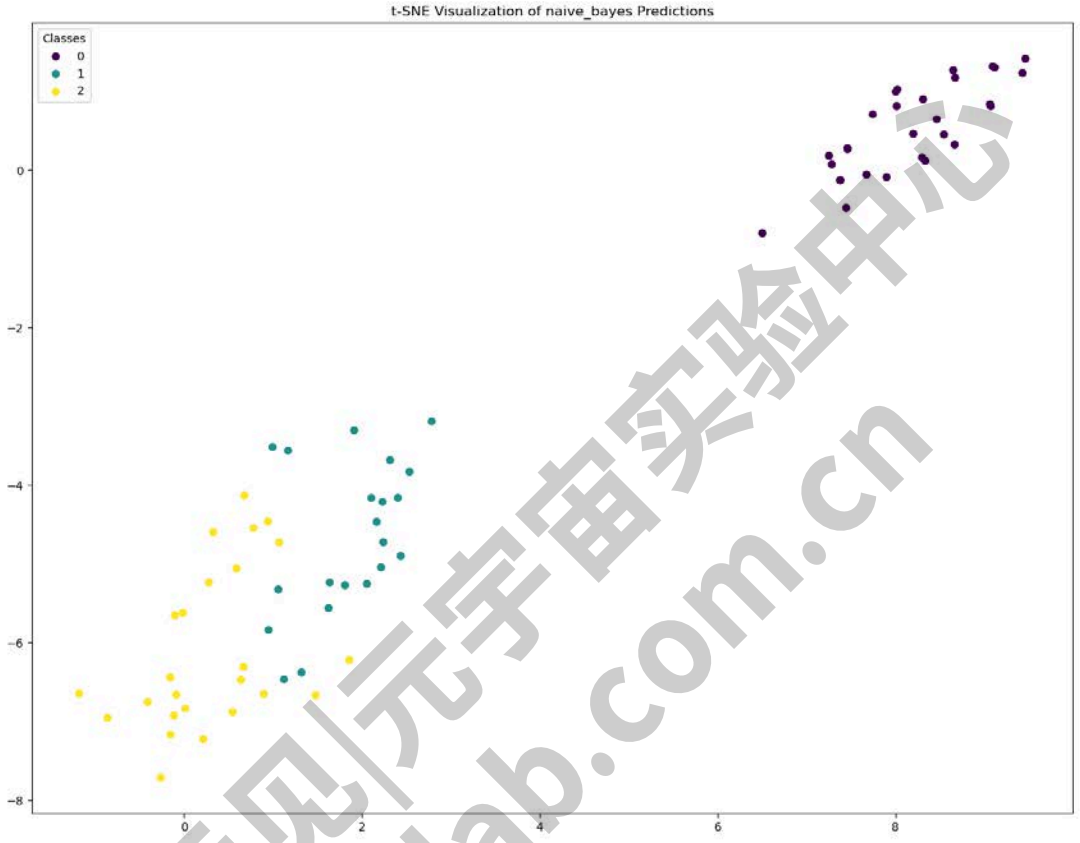
```
# 使用t-SNE进行降维
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X_test_scaled)

# 将字符串标签转换为数值标签，方便之后画图
label_encoder = LabelEncoder()
y_test_numeric = label_encoder.fit_transform(y_pred)

# 创建散点图进行可视化
plt.figure(figsize=(8, 6))
```

华清远见|元宇宙实验中心 yyzlab.com.cn

```
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1],
c=y_test_numeric, cmap='viridis')
plt.title('t-SNE Visualization of naive_bayes Predictions')
plt.legend(*scatter.legend_elements(), title='Classes')
plt.show()
```



华清远见元宇宙实验中心
yyzlab.com.cn